

## Indefinite Postponement in Multi-User Database Environments

### Part A – Conceptual Understanding

Indefinite Postponement, aka starvation, happens when a transaction is perpetually delayed and never gets its needed resource because other transactions keep winning the race for that resource. Unlike deadlock, also known as mutual waiting, starvation can lead to transaction being continually bypassed even though the system as a whole makes progress.

Yes, it is possible in practice for one user to block another indefinitely, depending on the locking/scheduling policy and system behavior described in following points:

1. Under strict locking schemes (two-phase locking), a transaction must hold locks until it commits or rollbacks. If a scheduling policy always favors new or high priority transactions, a low priority one might never acquire the lock it needs simply because others continuously acquire conflicting locks first.
2. A single long-running transaction that repeatedly reacquires resources can prevent others from making progress.
3. An “unlucky” transaction may be postponed forever also if no timeouts or deadlock breakers exist.

### Part B – Scenario Analysis

For the scenario, let us assume that there exist a table named *AccountTransaction* with rows A and B, transactions *T1* and *T2*, with operations as:

- *T1: update AccountTransaction set amount=amount-100 where id=A; then later reads B.*
- *T2: update AccountTransaction set amount=amount-50 where id=B; then later reads A.*

With strict exclusive row locks, lock manager uses non-FIFO selection favoring newer transactions or higher priority.

Now, let us look at the timing and locking sequence where the problem actually unfolds:

1. T1 starts, locks row A obviously.
2. Asynchronously (almost simultaneously), T2 starts and locks row B.

3. T1 now attempts to lock B but is waiting for T2 to finish.
4. T2 also attempts to lock A and is waiting for T1 to finish.

At this point, it is a classic and scary deadlock where both the transaction wait and they wait forever if the system has no deadlock detection implemented. But if the system has deadlock detection, it will pick a “victim” and abort one, avoiding starvation.

### **Part C – Systemic Impact Scenario**

Suppose, there is a long running transaction (dominant) that performs operation on a large range of rows and locks many pages (bulk update or something), and the DBMS uses strict 2PL and grants exclusive locks.

Now, short transactions arrive and request locks on rows held by dominant user/transaction and the lock manager queues them. But the system is configured to deprioritize queued short transactions (say to favor background and/or system workloads), or dominant transaction periodically renews locks or runs in high priority class, resulting in queue to never being empty.

This simply results in all short transactions being indefinitely blocked until dominant transaction finishes (starving every other users and/or transactions).

### **Part D – Reflection and Mitigation**

To prevent or mitigate indefinite postponement in database systems, the following strategies can be helpful:

#### **1. Deadlock detection/prevention**

Basically, the database management system builds a waiting graph and when a repeating cycle is detected, it chooses a victim transaction with less cost/progress to abort. This is best for resolving mutual deadlocks and prevents permanent mutual blocks between small groups.

#### **2. Fair scheduling**

Lock manager in a DBMS grants locks in order and ensures first arrived transaction is served first. It is best for preventing starvation due to unfair lock granting. All that said, FIFO can still hurt throughput in some workloads as it cares less about throughput than fairness.

### 3. Transaction timeouts

In this strategy, transactions that wait longer than a threshold are aborted and retried. This is best suited for breaking live locks and preventing indefinite waits. But poorly chosen timeout/retry policies can increase abort and retry rates, resulting in lower throughput.

### 4. Priority aging

Priority aging works by gradually increasing the priority of waiting transactions over time so they eventually get served. Even though it is best for preventing long term starvation in priority systems, it is limited due to its additional complexity and may slightly reduce short term throughput.